African Scientific Research and Innovation Council

BAAI 智源研究院

北京大学 PEKING UNIVERSITY

中国人民大学 RENMIN UNIVERSITY OF CHINA

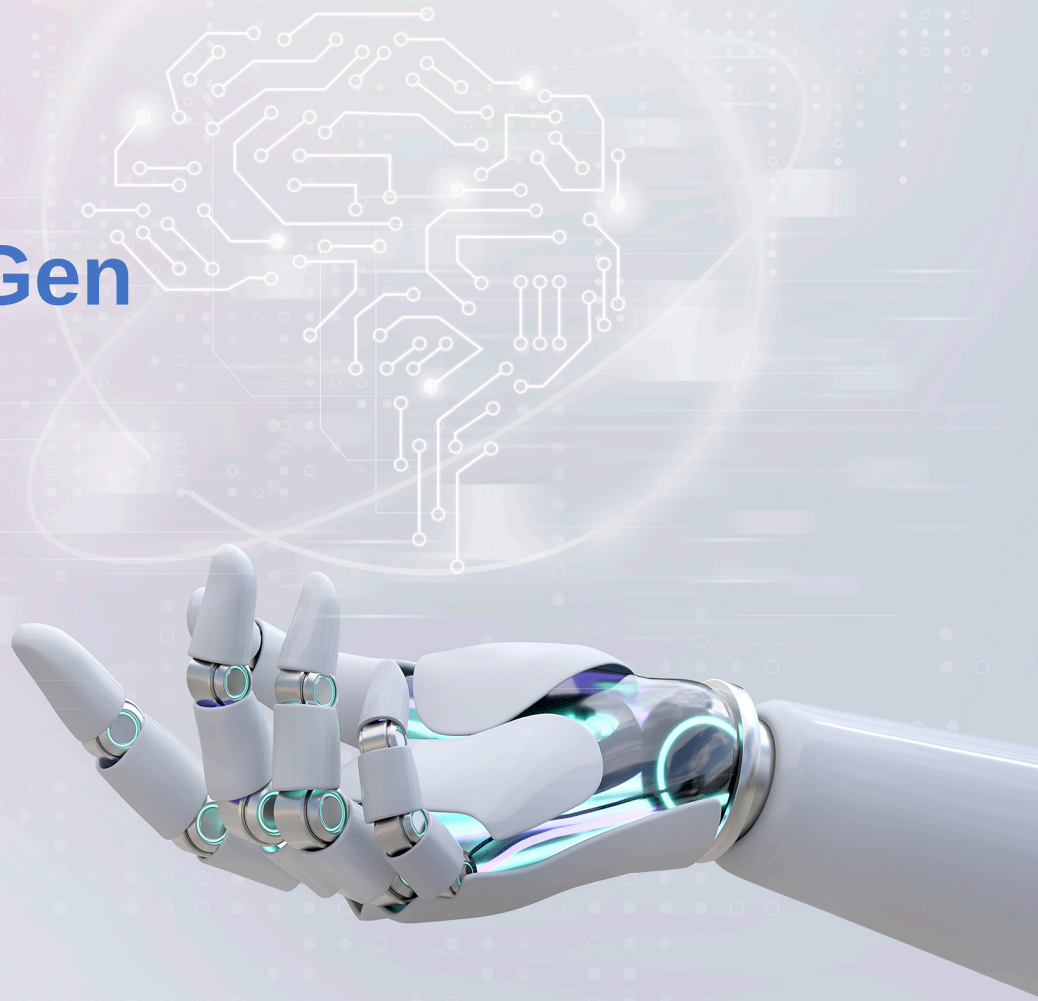
Global AI Compute Faculty Development Initiative

Module 05: Performance Benchmarking and Next-Gen Kernel Generation

Guojie Luo
罗国杰
gluo@pku.edu.cn

Jing Wang
王晶
jwang@ruc.edu.cn

October 2026 · 2nd AICFD



Supported by UNDP.

Outline

1. **Benchmarking & Performance Metrics**
2. **Profiling & Roofline Analysis**
3. **Auto-Tuning & KernelGen**
4. **End-to-End Optimization**
5. **Lab: Performance Analysis in Practice**

1. Benchmarking & Performance Metrics

- Why Performance Metrics?
- Classification of Metrics
- FLOPS, Bandwidth, Latency & Utilization
- MLPerf Industry Standard

1.1. Why Do We Need Performance Metrics?

In High-Performance Computing (HPC) and system optimization,

we need to **quantify** program and hardware performance — different metrics reveal bottlenecks at different levels

Common Performance Metrics

- **FLOPS** Compute Capacity
- **Bandwidth** Bandwidth
- **Latency** Latency
- **Utilization** Resource Utilization

> **Roofline Model**
> Performance Ceiling Model

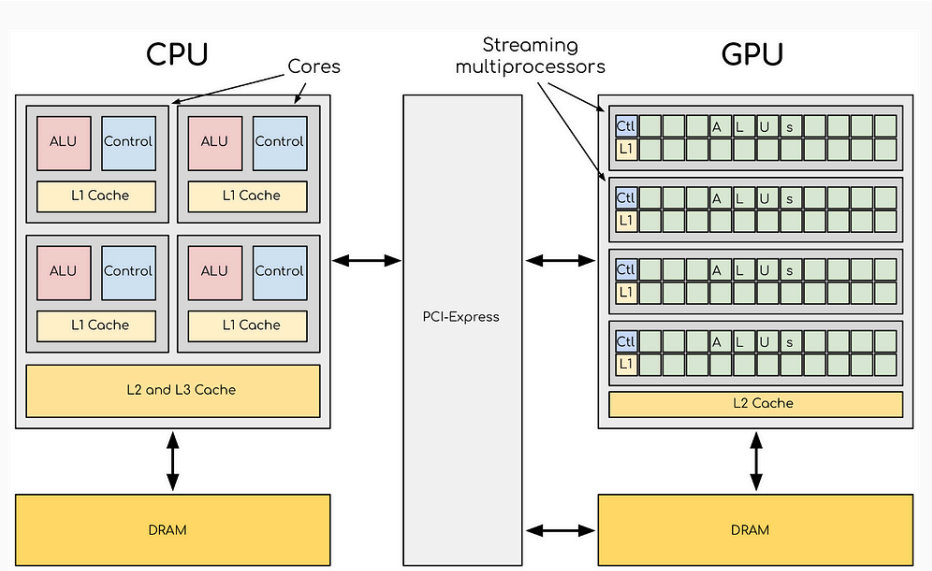


Image Source: ENCCS — GPU Ecosystem

1.2. Classification of Performance Metrics

Compute Capacity
Compute Bound

FLOPS

Measures the number of floating-point operations a processor can execute per second — the core metric for compute-intensive task performance.

Utilization

Reflects the actual usage efficiency of hardware resources, helping identify whether compute units are fully utilized.

Data Access Capability
Memory Bound

Bandwidth

Represents the amount of data transferable per unit time, determining throughput capacity for data-intensive applications.

Latency

Represents the time required to complete one data access, directly affecting program responsiveness and real-time performance.

Roofline Model used to analyze both bottleneck types, helping determine whether a program is compute-bound or memory-bound

1.3. What is FLOPS ?

FLOPS = Floating Point Operations Per Second

Assumed CPU Specs

Clock Frequency	FLOPs per Cycle
3 GHz	**8 FLOP**

Theoretical Performance Calculation

$$\text{FLOPS} = \text{Frequency} \times \text{FLOPs per Cycle}$$
$$= 3 \times 10^9 \times 8$$
$$= \mathbf{24\ GFLOPS}$$

Note: This is the theoretical peak; actual performance is typically lower

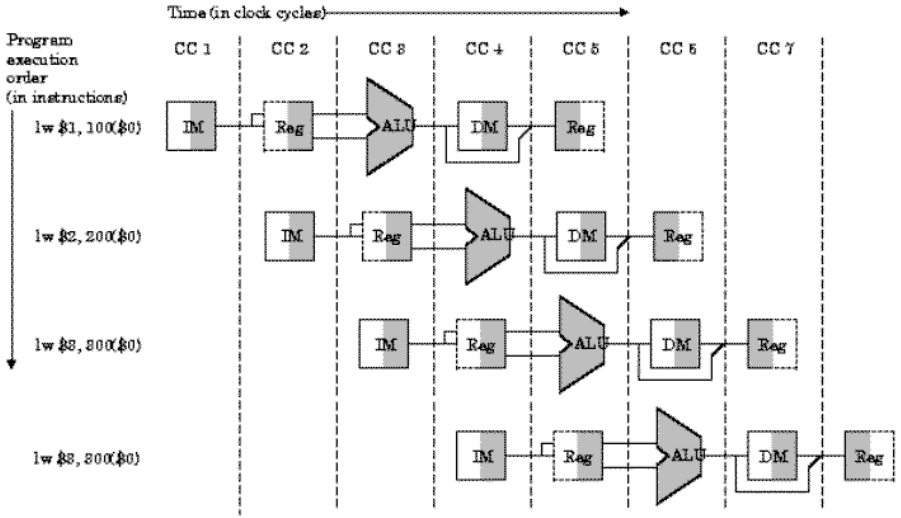


Image source: University of Florida - CPU Pipeline

1.4. Bandwidth

Bandwidth
Represents the amount of data transferable per unit time

Common Units

MB/s Megabytes/second

GB/s Gigabytes/second

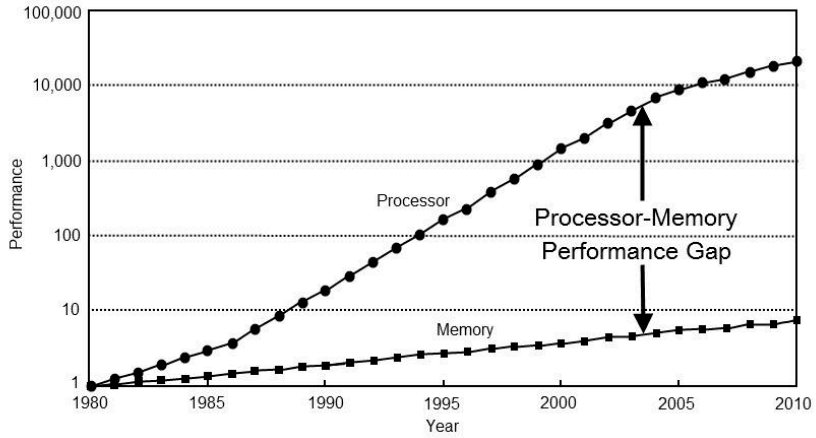
TB/s Terabytes/second

Use Cases

Memory Bandwidth

Disk Bandwidth

Network Bandwidth



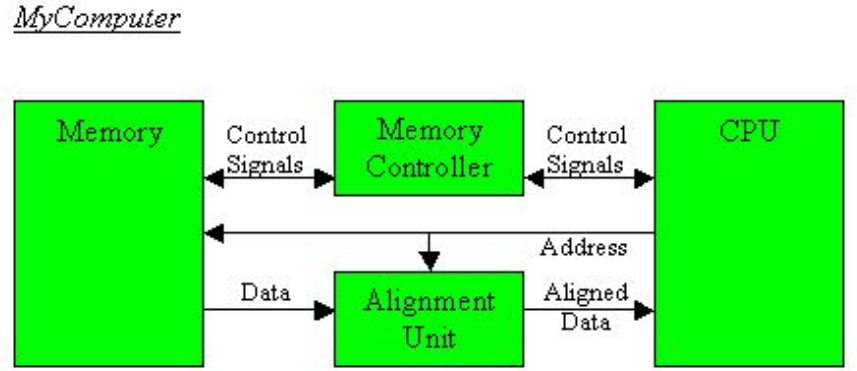
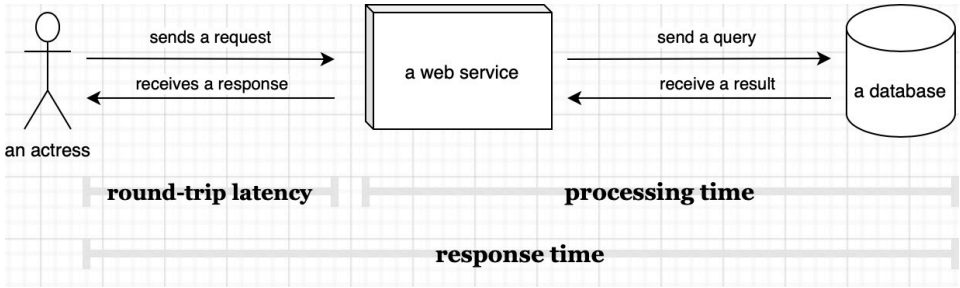


Image source: ForrestTheWoods — Memory Bandwidth; University of Pittsburgh — Memory Controller

1.5. What is Latency?

Latency
Represents the time required to complete one operation



Typical Scenarios

Memory Access Latency	Network Latency	Disk Access Latency

Common Units

[ns]
Nanosecond
10^{-9} second

[µs]
Microsecond
10^{-6} second

[ms]
Millisecond
10^{-3} second

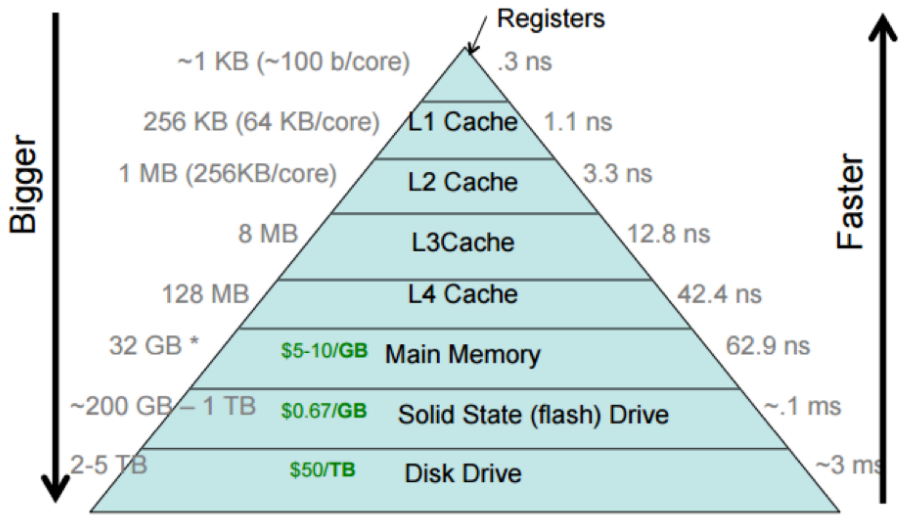


Image source: Scalable Developer - Latency Timeline; Brown University - Memory Hierarchy

1.6. Utilization

Actual usage level of hardware resources

Even with powerful hardware, if utilization is low → performance will still be poor

Formula

Utilization = Actual Performance / Theoretical Peak

Typically expressed as a percentage

Calculation Example

GPU Theoretical	Actual Program	
10 TFLOPS	**5 TFLOPS**	**50%**



Image source: Grafana — GPU Metrics Dashboard



Image source: Lambda Labs — GPU Monitoring Tools

1.7. MLPerf – Industry Standard for ML Performance

What is MLPerf?

An industry-standard benchmark suite for machine learning performance

Two main suites: MLPerf Training and MLPerf Inference

Over 3,500 individual results from 100+ system configurations (as of 2021)

Performance spans 5 orders of magnitude – from tiny embedded devices to cloud-scale datacenters

Why MLPerf matters?

Provides fair, reproducible comparison across hardware and software

Drives performance improvements – 1.3× speedup in 16-chip results between v0.5 and v0.6 despite higher quality targets

Growing adoption: 3 submitters (40+ results) → 14 submitters (500+ results) in early rounds

Key design principles

Closed Division: Fixed model for apples-to-apples hardware comparison

Open Division: Any model allowed to encourage innovation

Time-to-train as the primary metric for training (vs. throughput)

Semantic-level benchmarks: Submitters reimplement models for their hardware

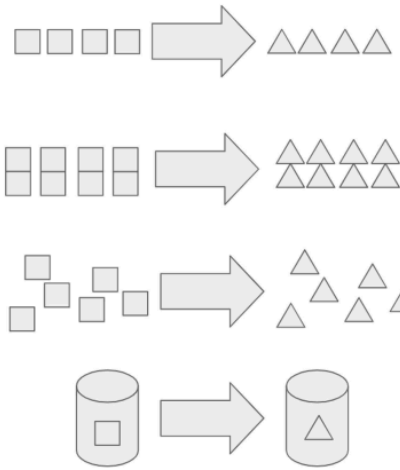
1.8. MLPerf Inference – Scenarios & Metrics

Quality targets:

- Most tasks: within 1% of FP32 reference accuracy
- Medical imaging / recommendation: within 0.1% (real-world stringency)

Performance varies drastically across scenarios:

- Server vs Offline throughput ratio: up to 0.55× (GNMT), 0.8× on average
- Systems optimized for throughput may perform poorly under latency constraints



Single stream e.g. cell phone augmented vision	**Metric: latency**
Multiple stream e.g. multiple camera driving assistance	**Metric: number of streams** subject to latency bound
Server e.g. translation site	**Metric: QPS** subject to latency bound
Offline e.g. photo sorting	**Metric: throughput**

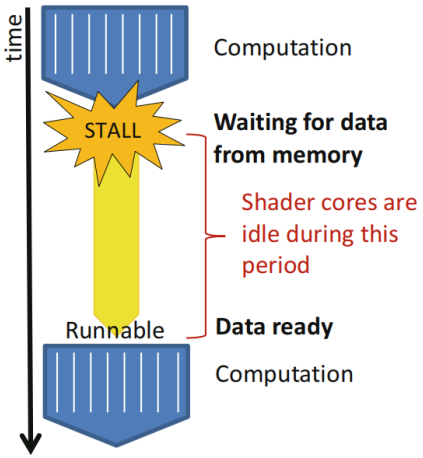
Reddi, V. J., Cheng, C., Kanter, D., Mattson, P., Schmuelling, G., & Wu, C.-J. (2021). The vision behind MLPerf: Understanding AI inference performance. IEEE Micro, 41(3), 10–18.

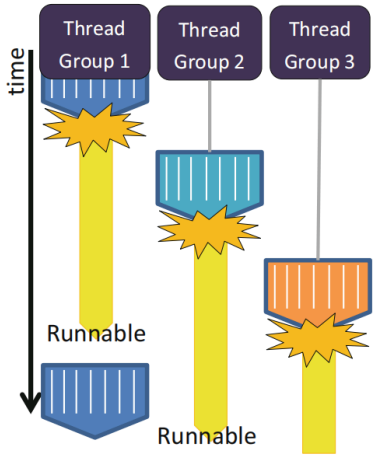
2. Profiling & Roofline Analysis

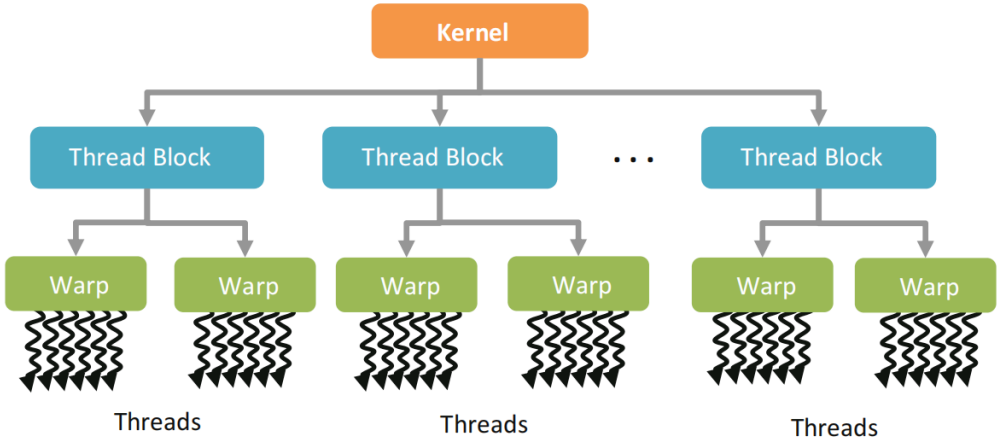
- Profiling Answers (GPU Execution)
- GPU Occupancy
- Profiling Tools (BI-V150, ixKN, Proton)
- Roofline Model

2.1. Profiling answers: Where is the time spent inside the kernel?

- Execution model under GPU Architecture
  - GPU execution without/with batched thread groups
  - Single-instruction multiple threads (SIMT)
  - Hierarchical thread execution model







2.2. Occupancy – Keeping the GPU Busy

Occupancy = Active Warps / Maximum Warps per SM

Three limiting factors:

Registers
Each SM has limited registers (e.g., 16384)

Shared Memory
Limited capacity per SM

Thread Block Size
Hardware limit (NVIDIA: 1024, BI-V150: 4096)

Example: GPU with 16384 registers/SM, kernel needs 35 registers/thread

- Each SM can execute at most 468 threads
- Block size 512 is not possible; only 1 block of 256 allowed (212 threads idle)
- 3 blocks of 128 threads → 384 schedulable threads

2.3. Occupancy – Limiting Factors

The minimum of these three factors limits the active number of threads (or occupancy) of a compute unit



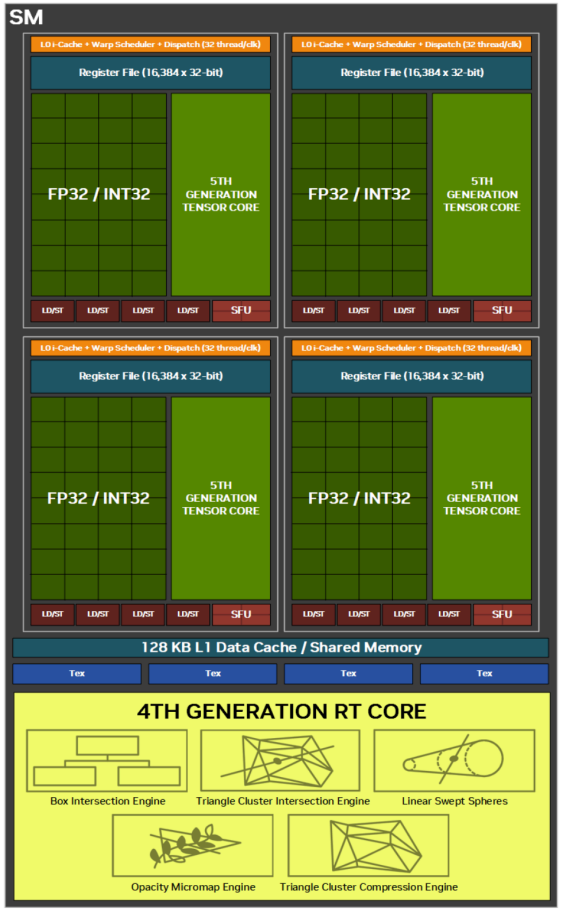
Interactions are complex

- Limiting factor may have thread or wavefront granularity
- Changing work-group size may affect register or shared-memory usage
- Reducing any factor slightly may allow another work group to be active

The Nsight occupancy calculator from NVIDIA plots these factors visually, allowing tradeoffs to be visualized

2.4. Occupancy: Tradeoffs

- Shared memory and registers are persistent within SM when other warps execute
  - Allows for lower overhead context switch
- The number of active warps that can be supported per SM is limited
  - Decided by the shared memory per block and register usage per thread
- The number of active warps possible on an SM can be expressed using a metric called utilization
- Larger numbers of active warps allow for better latency hiding on both NV and AMD hardware



2.5. Profiling Tools on BI-V150 (Lab Platform)

[ixSMI: System-level monitoring]

`ixsmi`
View GPU status (temp, power, memory)

`ixsmi dmon`
Real-time device monitoring

`ixsmi pmon`
Process monitoring

[ixSYS: System-wide trace capture]

`ixsys -t cuda,nvtx -o output.trace ./program`
Captures CUDA API calls & kernel execution

Web-based visualization UI

[ixKN: Kernel-level performance analysis]

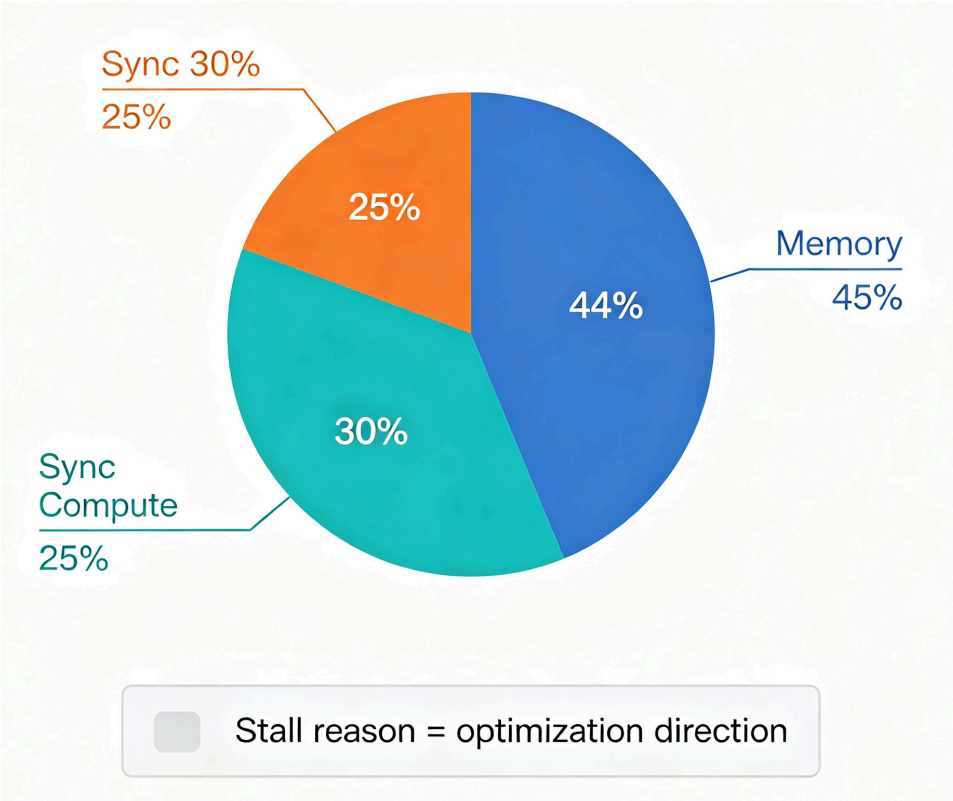
`ixkn-cli –set default ./program`
Kernel profiling entry point

Sections
Instruction · LaunchStats · Occupancy · WarpStateStats · Memory

2.6. ixKN – A Deep Dive into Kernel Performance

Section	What It Tells You
LaunchStats	Grid/Block config, resource usage
Occupancy	Active warps vs. maximum, utilization rate
VarpStateStats	Why warps are stalling (memory, sync, compute)
Instruction	Instruction mix and execution counts
Memory	Shared memory bank conflicts, cache hit rates

WarpStateStats example

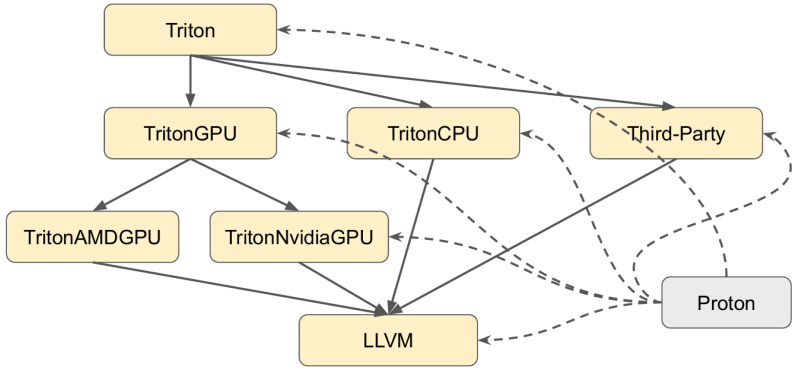


2.7. Proton – A Multi-Level Profiler for Triton

What is Proton?

- Multi-level profiling: full model → layers → Triton ops → compiler IR
- Custom metrics: FLOPs, Bytes, Tokens beyond hardware counters
- Low overhead: ≤1.5× runtime, <1MB profile size
- Multi-vendor: NVIDIA (CUPTI) and AMD (roctracer)
- Programmatic & CLI: `python -m proton` or `proton.scope()`

Proton Dialect



What metrics does profiling care about?

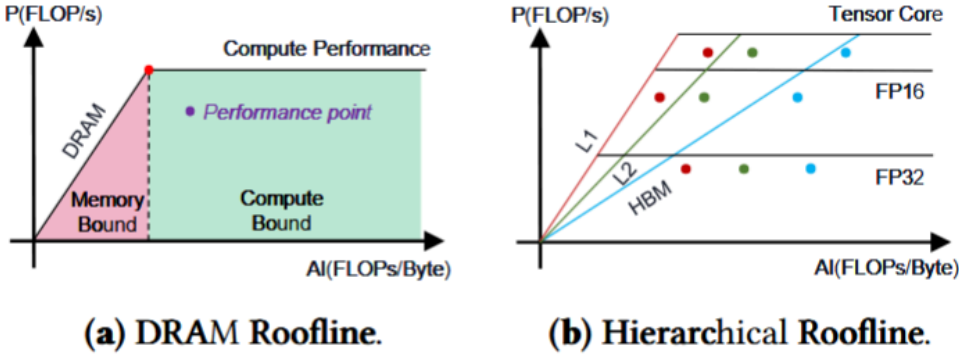
- Kernel execution time
- Instruction samples — expensive lines / IRs / instructions
- Occupancy & utilization
- Memory stall reasons (shared / global)
- User-defined metrics (FLOPs, Bytes, Tokens, …)

Zhou et al., “Proton: Towards Multi-level, Adaptive Profiling for Triton,” CGO 2026.

2.8. The Roofline Model – Visualizing Performance Limits

A visual performance analysis model that answers:

- Is my program compute-bound or memory-bound?
- How far am I from the hardware limit?



(a) DRAM Roofline. (b) Hierarchical Roofline.

Figure 2. Existing roofline models.

Two constraints

- Compute ceiling (horizontal): $P \leq \pi$ (peak FLOP/s)
- Bandwidth slope (diagonal): $P \leq \beta \times I$

Ridge point

- Left of ridge → Memory-bound
- Right of ridge → Compute-bound

Y. Zhou et al., “Squeezing Operator Performance Potential for the Ascend Architecture,” ASPLOS 2025.

2.9. Beyond DRAM – Hierarchical & Component-Based Roofline

Hierarchical Roofline

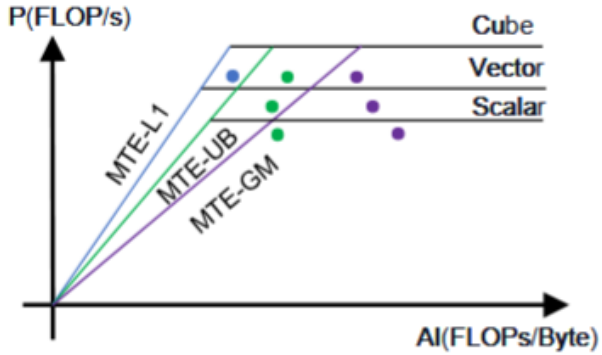
- Multi-level caches (L1, L2, DRAM) — any layer can bottleneck
- Also supports multiple precision/compute units (FP32, FP16, Tensor Core)

Challenge: Roofline on DSAs (Ascend)

- Unique compute units and memory transfers
- Naive: rooflines for every precision-transfer combo → 180 combinations

Component-Based Roofline Model

- ASPLOS 2025: abstract components → identify bottleneck components
- Instead of 180 combos → component-level bottlenecks
- Results: 1.07×–2.15× end-to-end speedups across 11 tasks



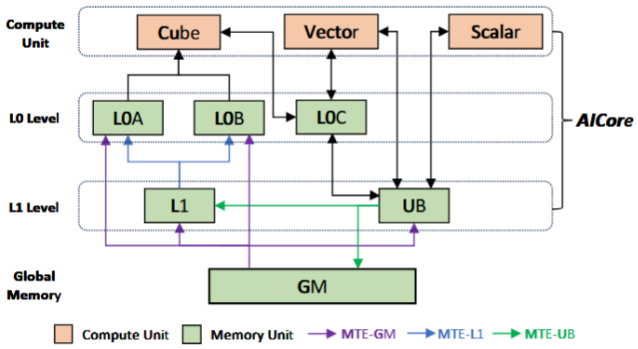


Figure 1. The architecture of AICore in Ascend.

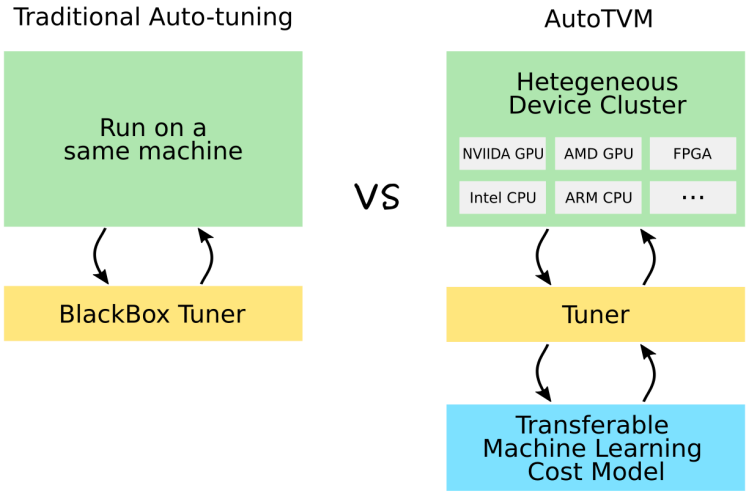
Y. Zhou et al., “Squeezing Operator Performance Potential for the Ascend Architecture,” ASPLOS 2025, pp. 1156–1171.

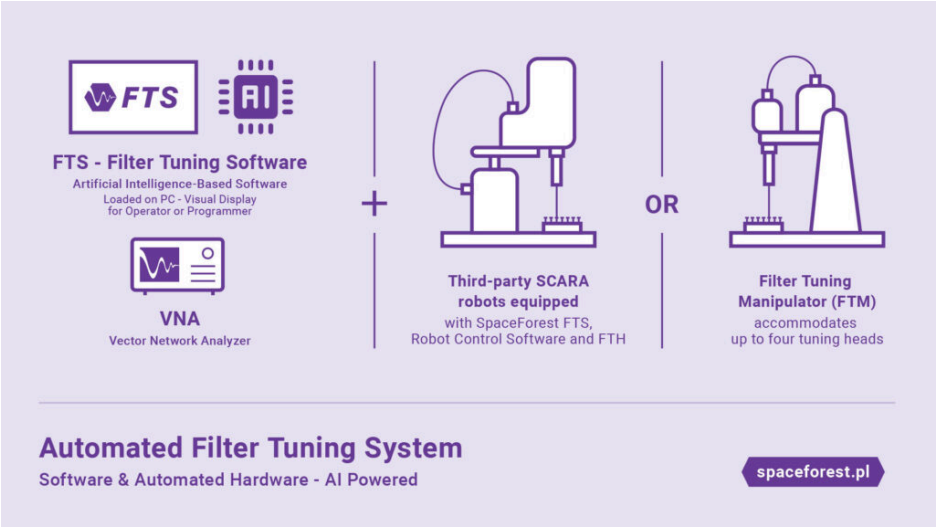
3. Auto-Tuning & KernelGen

- Automatic Tuning
- What is Kernel Auto-tuning?
- Application of Simulated Annealing
- Advanced Exploration Algorithm: Genetic Algorithm
- Advanced Techniques
- KernelGen background && workflow

3.1. Automatic Tuning

- Automatic Tuning Algorithm and Underlying Implementation
  - From Template Search to AI-Driven Operator Tuning

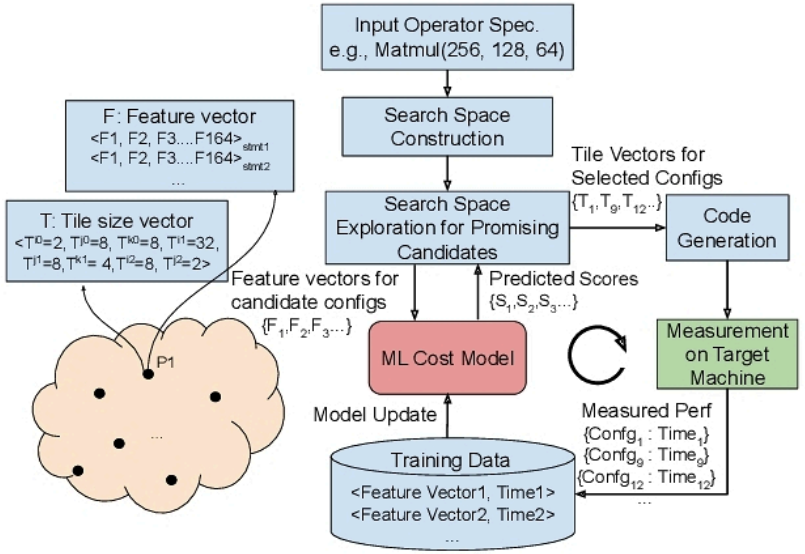




3.2. What is Kernel Auto-tuning?

Automation of Code Generation

- Input: Mathematical calculation expressions (e.g., tensor multiplication, convolution)
- Output: Low-level code optimized for specific hardware (e.g., CUDA, LLVM IR)

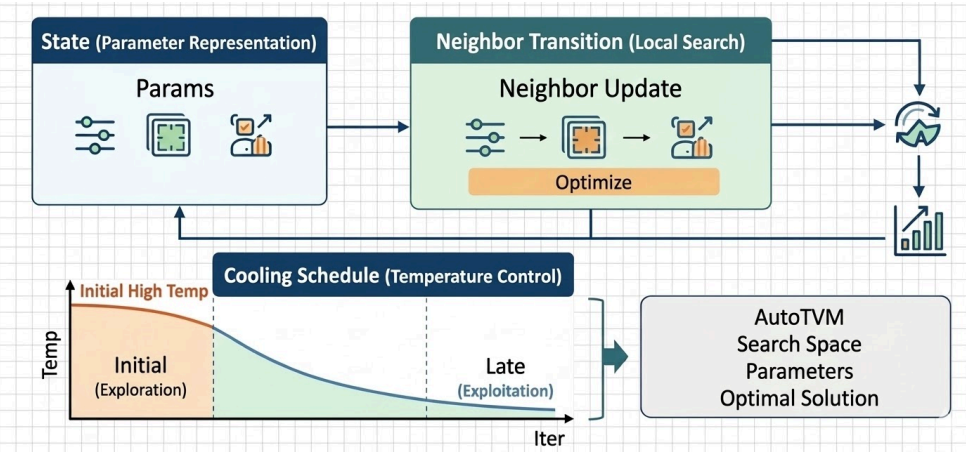


Common Tuning Dimensions (Schedule Primitives)

- Tiling: Improves cache hit rate
- Unrolling: Reduces branch overhead
- Vectorization: Utilizes SIMD instructions (e.g., AVX, TensorCore)
- Binding: Block/Thread mapping in the GPU

3.3. Application of Simulated Annealing

- **State Representation:** The current set of template parameters
- **Neighbor Transition:** Fine-tuning one of the parameters
- **Cooling Schedule:**
  - Initial High Temperature: High probability of accepting poor solutions (large-scale exploration)
  - Later Low Temperature: Almost only accepting good solutions (local convergence)
- Early versions of AutoTVM widely used simulated annealing to find optimal parameters in the candidate space

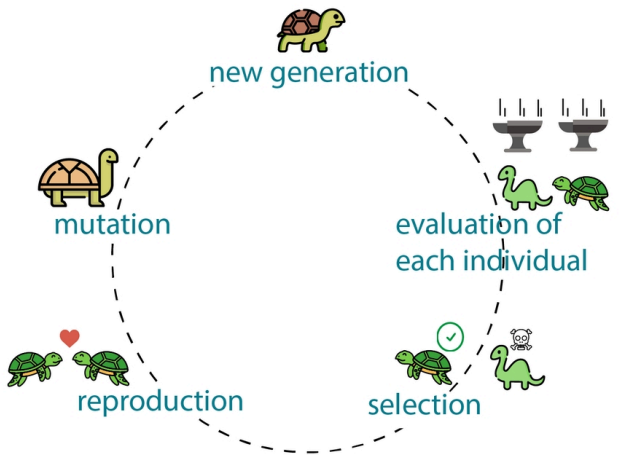


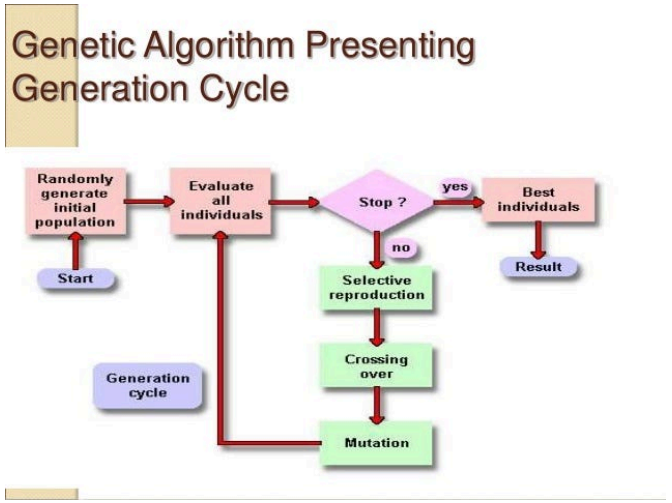
3.4. Advanced Exploration Algorithm: Genetic Algorithm

Inspiration: Darwin's theory of evolution, "survival of the fittest."

Core Algorithm Steps

- Population: Randomly generate a batch of initial configurations
- Fitness Evaluation: Evaluate performance using a cost model
- Crossover: Swap parameters between superior configurations
- Mutation: Randomly change parameters (prevent premature convergence)
- Selection: Eliminate poor configs; retain better ones for next generation

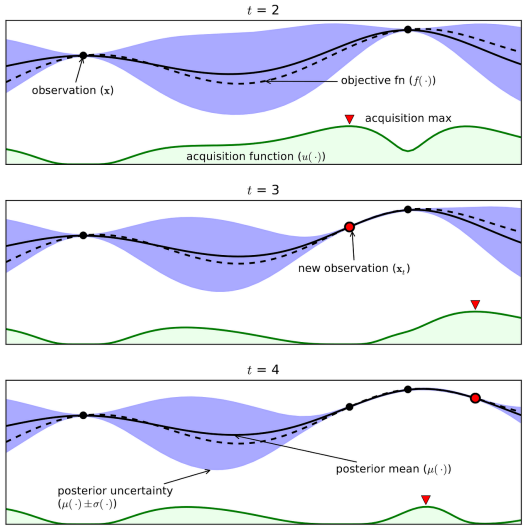




3.5. Advanced Techniques

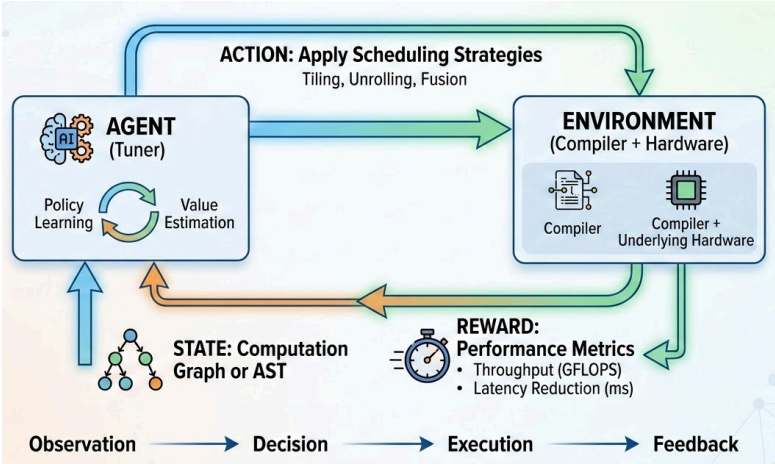
Bayesian

- Suitable Scenarios: Optimization of black-box functions where a single evaluation is extremely costly
- Core Components
  - Surrogate Model: Typically uses a Gaussian process
  - Acquisition Function: Often uses EI (Expected Improvement)



Reinforcement Learning

- Tuning from a Reinforcement Learning Perspective (MDP — Markov Decision Process)



3.6. KernelGen background && workflow

KernelGen core objectives

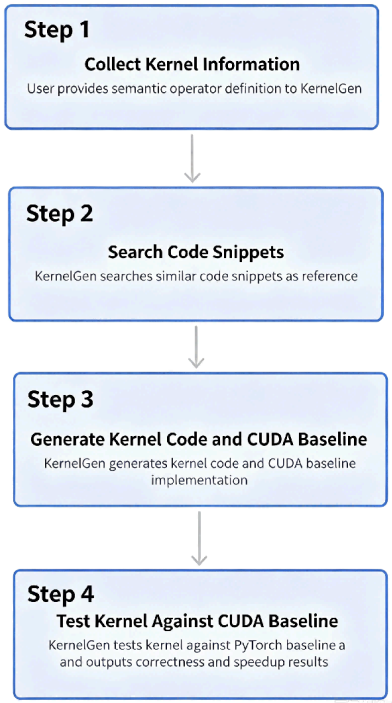
Utilizing AI technology to automatically generate GPU kernel code, achieving ultimate performance optimization for target hardware and resolving the challenges of manual programming.

The industry faces challenges

The GPU programming model is complex, with large differences between architectures, lacking general optimization strategies, and having extremely high thresholds for parallel computing scheduling and memory management.

Core value of the project

Significantly lowering the threshold for GPU application development, unleashing the potential of hardware computing power, and improving the execution efficiency of computing tasks as well as resource utilization.



3.7. KernelGen Skills Route the Task

- KernelGen generates, optimizes, and specializes Triton kernels. The MCP toolkit does the kernel work; the `kernelgen-flagos` skill is the procedure the agent must follow.
- Route first. Detect FlagGems, vLLM, or a general repository, then dispatch generate, optimize, or specialize. Do not mix those procedures.
- The agent does not write the Triton kernel. Generation calls `generate_kernel`, optimization `optimize_kernel`, and specialization `specialize_kernel`. If MCP fails, stop.

flagos-ai/KernelGen, https://github.com/flagos-ai/KernelGen. Skill: `kernelgen-flagos`.

3.8. Generate: Steps and flagos_wiki

- Fixed order: check torch, Triton, and MCP, then parse `kernel_name` and `func_type`, map the repository, and see whether the operator already exists. Only then call MCP.
- `flagos_wiki` is the knowledge handed to the generator: kernel pattern, “use Triton”, memory layout and access, and repository conventions such as wrappers and autotune configs. At most ten short hints.
- MCP returns a PyTorch reference, a Triton kernel, an accuracy test, and a benchmark. The skill places them, registers the operator, then reports pass rate and speedup.

3.9. Optimize: Versions the Skill Remembers

- Each try is appended to `optimization_log`: the action, whether it was correct, the speedup, and a status of applied, rolled back, or fixed then applied.
- Two copies stay current. `safe_version` is the last kernel that passed correctness. `best_version` is the fastest correct one. Both are backed up as files, so a rollback does not depend on chat memory.
- Correctness is a short loop, at most 3 tries. Performance is a longer loop, default 10. A failed candidate returns to the safe kernel.

4. End-to-End Optimization

- End-to-End Collaborative Optimization
- Double Walls of LLM Inference
- Compiler-Level Optimization
- Operator Implementation
- FlagOS Ecosystem
- vLLM-Plugin-FL Design
- FlagGems + FlagTree

4.1. What is end-to-end collaborative optimization?

Limitations of local optimization

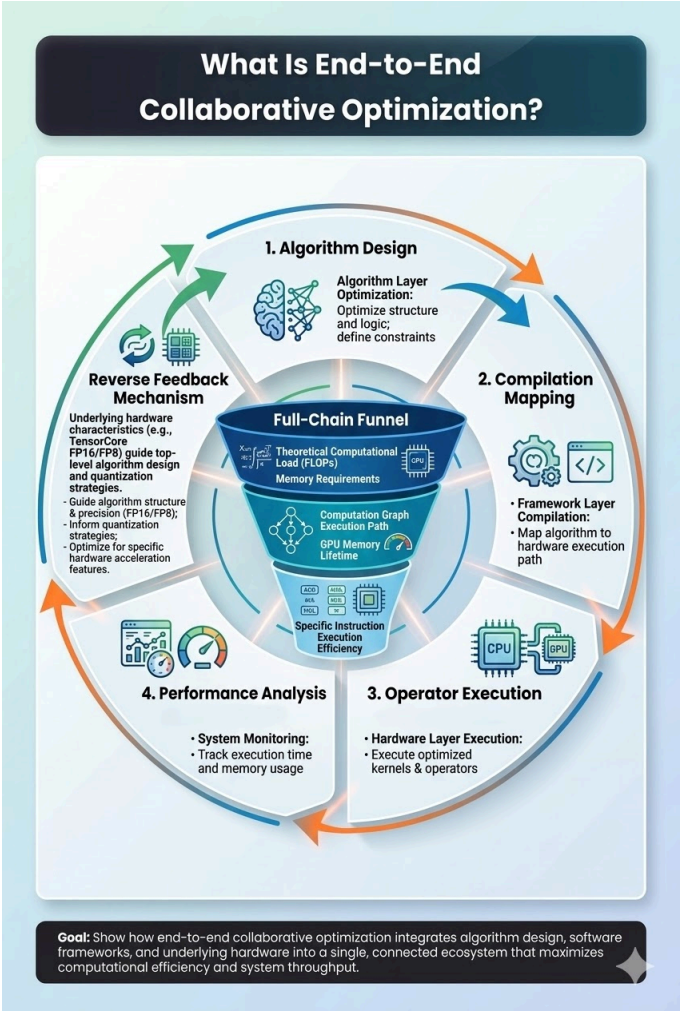
- Amdahl's Law

Full-chain funnel model

- Top (algorithm): theoretical FLOPs and memory requirements
- Middle (compilation/framework): computation-graph path and GPU-memory lifetime
- Bottom (operator/hardware): execution efficiency of specific instructions

Reverse feedback mechanism

- Underlying hardware traits (e.g., TensorCore FP16/FP8) guide top-level algorithm design and quantization



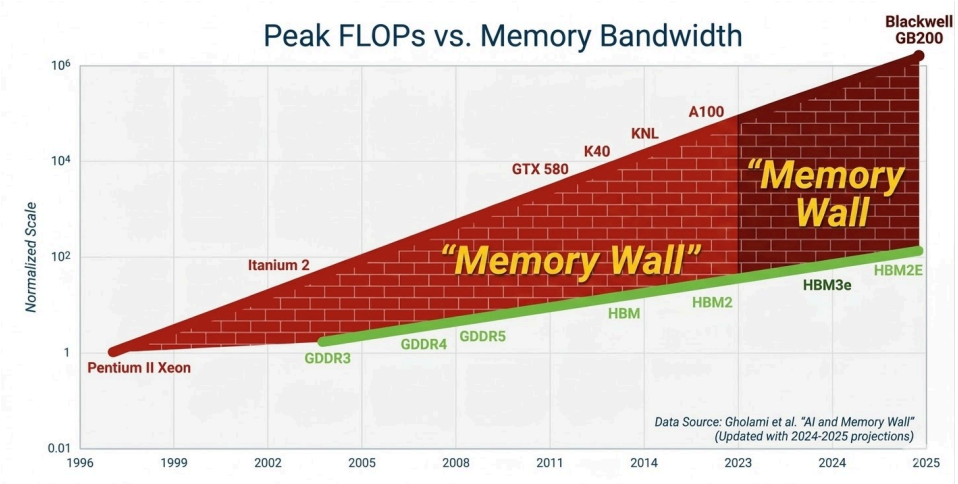
4.2. The “Double Walls” of Large Model Inference

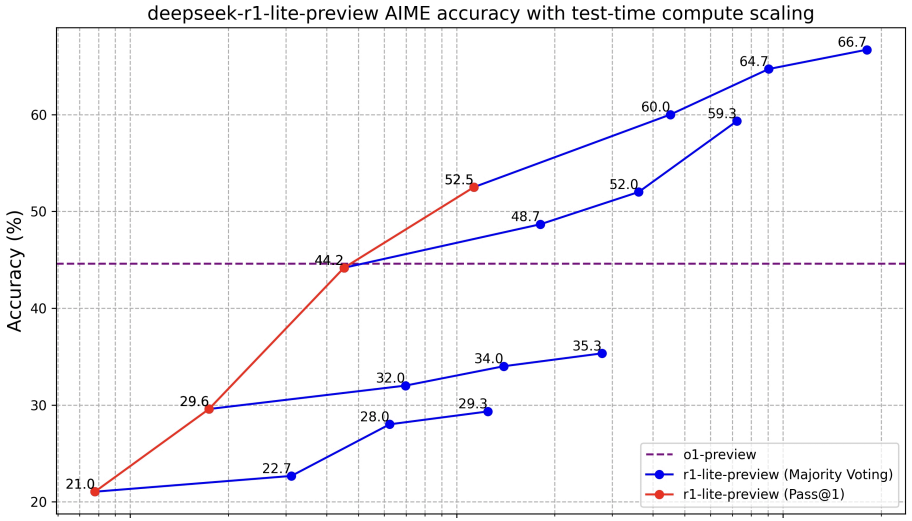
Memory Wall

Decoding is primarily limited by memory-access bandwidth. Massive KV cache consumes GPU memory, limiting concurrent batch size.

Compute Wall

Prefill with long contexts: attention cost grows quadratically with sequence length (compute-bound).

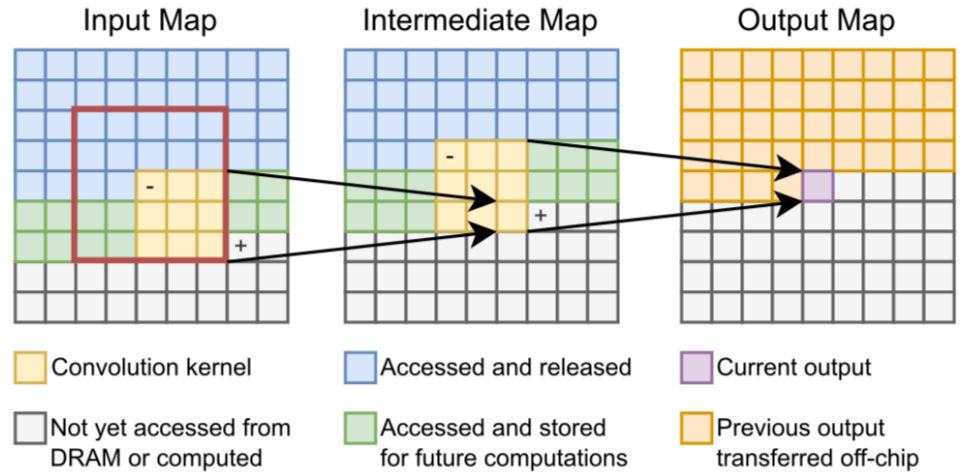


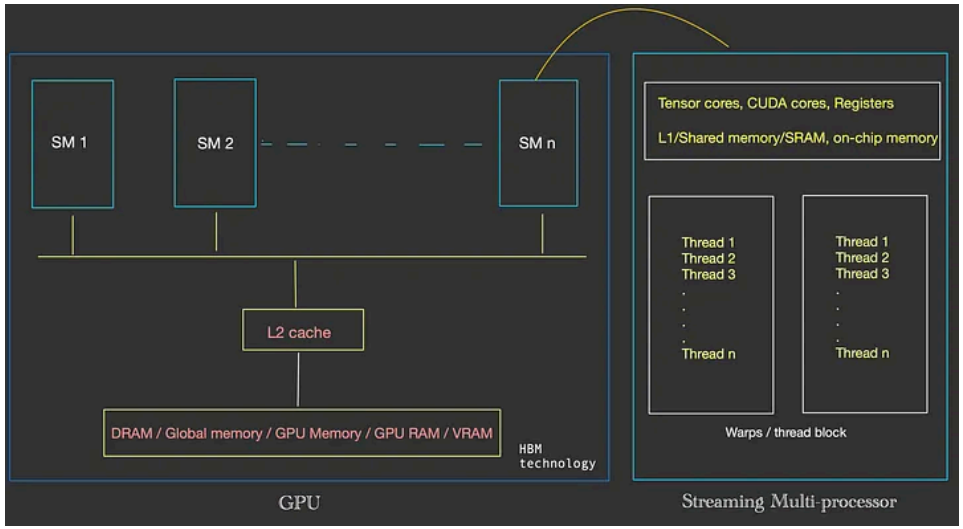


4.3. Compiler-Level Optimization Strategies

Graph Optimization

- **Operator Fusion:** Merge small operators (e.g., MatMul + Bias + ReLU) into one kernel
- **Benefits:** Avoid repeated DRAM ↔ SRAM traffic for intermediates; keep data in registers/shared memory
- Constant Folding & Dead Code Elimination





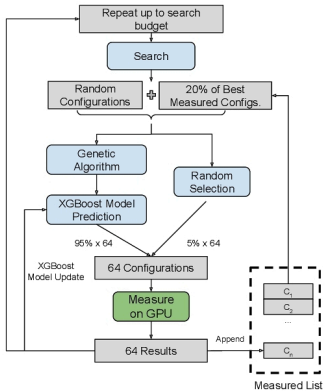
4.4. Operator Implementation

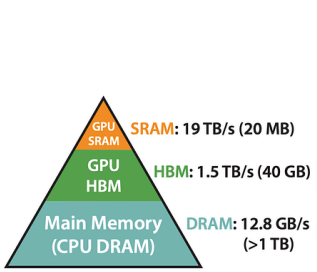
Following up on Automatic Tuning

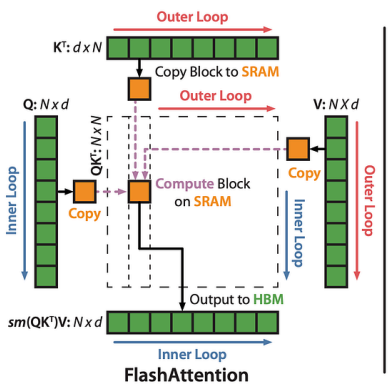
- Optimal schedule from AutoTVM / heuristics is compiled into final hardware instructions

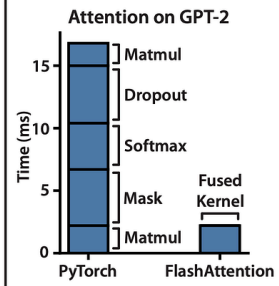
Core Optimization Techniques

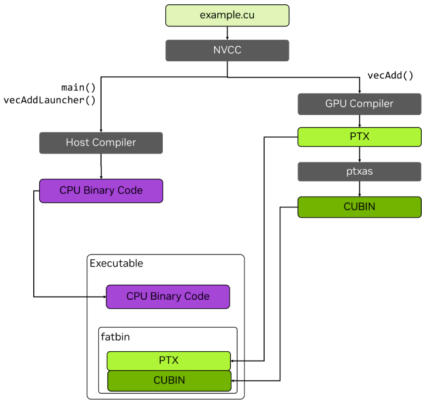
- FlashAttention and variants: tiling + recomputation avoid materializing the full Attention matrix
- Extreme assembly-level optimization: PTX or handwritten kernels for specific microarchitectures







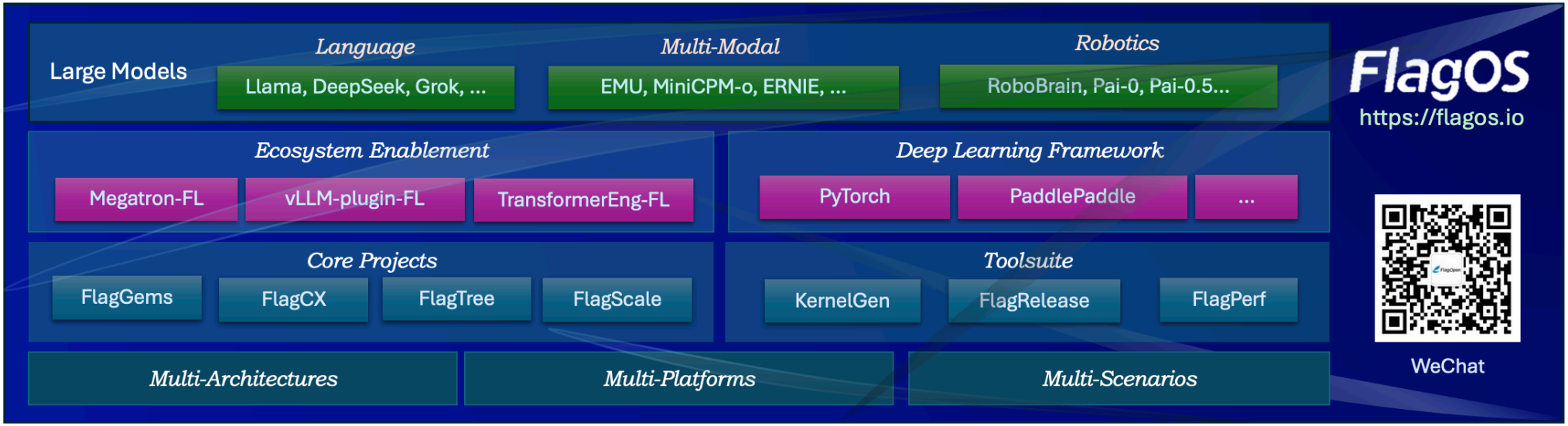




4.5. The FlagOS Ecosystem

What FlagOS does

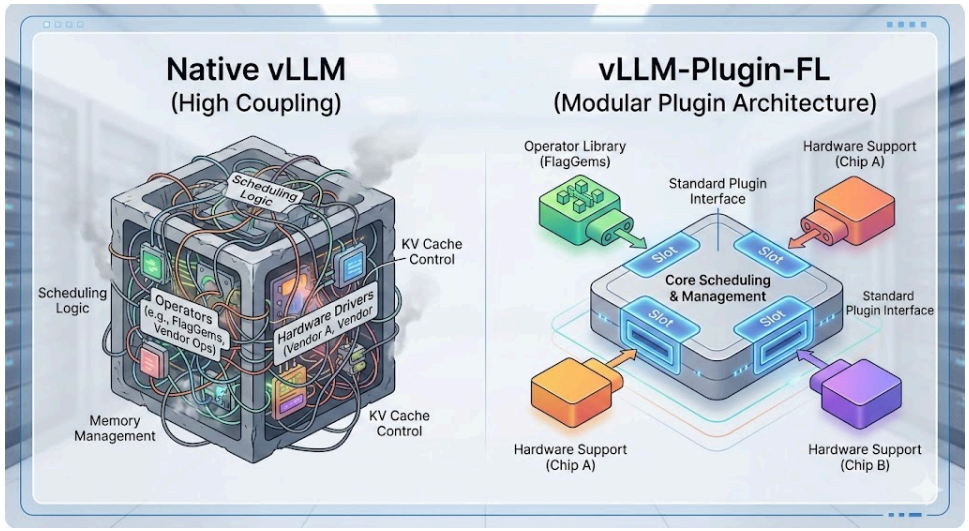
- Unified heterogeneous computing open-source software stack
- “Write once, run on any hardware”
- Supports NVIDIA, Days, Hygon, Huawei, Moore Threads

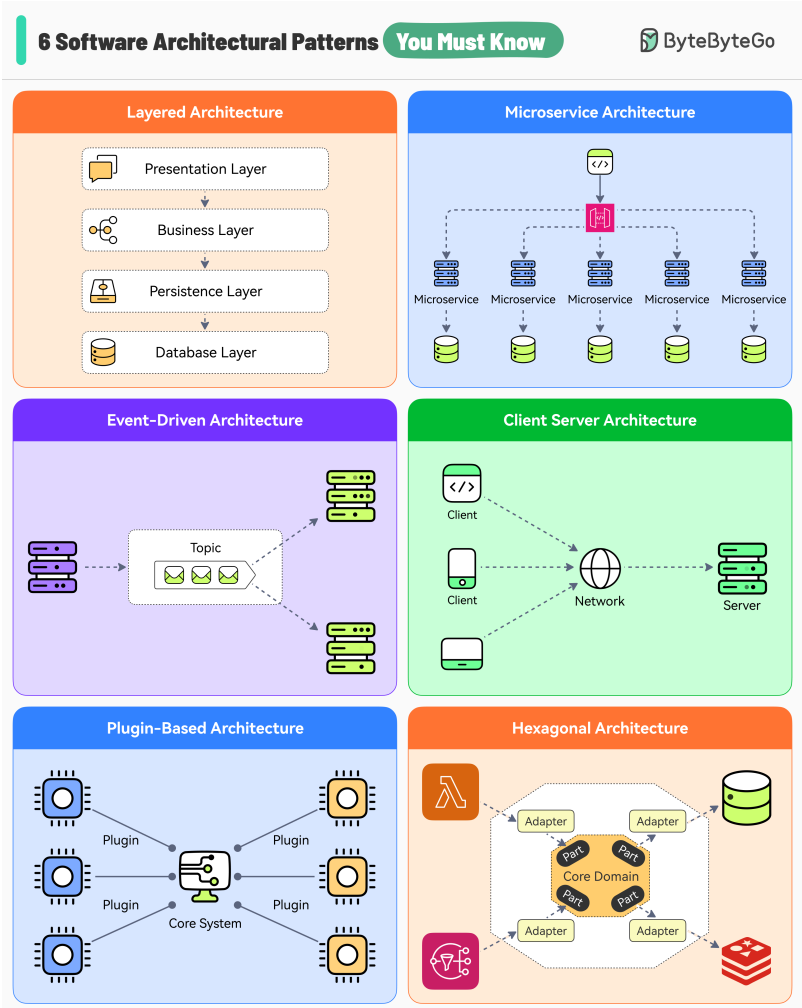


4.6. vLLM-Plugin-FL Design Philosophy

Plugin-Based Design

- **Decoupling:** Separate core scheduling from underlying hardware execution / operator libraries
- **Hot-Swapping:** Quick integration with self-developed optimization libraries such as FlagGems

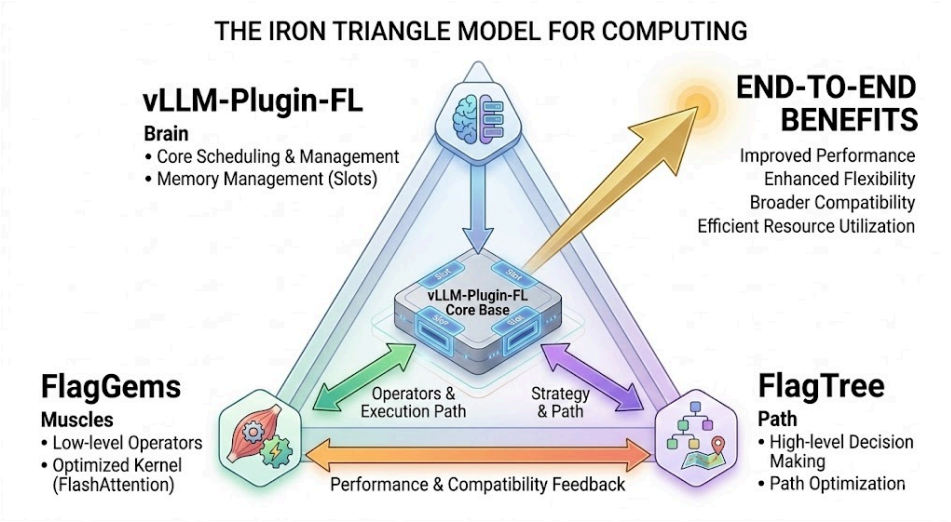




4.7. vLLM-Plugin-FL + FlagGems + FlagTree

The Iron Triangle Collaboration

- **vLLM-Plugin-FL:** Brain — request scheduling, KV Cache, computation-graph distribution
- **FlagGems:** Muscles — highly optimized kernels (FlashAttention, fusion ops)
- **FlagTree:** Higher-level optimization of decision trees / execution paths



End-to-End Benefits: “Out-of-the-box” high performance across multiple platforms

5. Lab: Performance Analysis

- Module 5: Performance Evaluation & Optimization Practice
- Three Experiments – What Question Does Each One Answer?

5.1. Module 5: Performance Evaluation & Optimization Practice

How to Run This Lab

- Lab 1 introduced how to write and tune a GEMM kernel
- This module teaches you how to prove whether your changes actually help — scientifically, and layer by layer

```
bash setup.sh && python3 00_smoke_cuda.py
export LAB_DAY2_ROOT=$PWD/../lab1-day2

python3 01_benchmark.py --scenario offline
python3 01_benchmark.py --scenario server
python3 02_roofline.py
python3 03_profiling.py
python3 04_summary.py   # then fill in REPORT.md
```

Repository

- Student repo: gitee.com/yihan-long/day2-lab5
- Target platform: BI-V150 / FlagOS; SUT = Day2 GEMM (or in-repo equivalent)

Environment smoke test
→ ① Benchmark → ② Roofline
→ ③ Profiling → Fill in REPORT.md

5.2. Three Experiments – What Question Does Each One Answer?

Step	What We Do	What We Measure / Look At	The Question It Answers
① Benchmark	Run GEMM repeatedly through LoadGen (Offline / Server)	Median/P99 latency, throughput, effective TFLOP/s; compare torch / baseline / autotuned	How fast is it? How large is the gap?
② Roofline	Plot the benchmark results against hardware peak values	Arithmetic intensity (I), ridge point, position relative to the ceiling	Is the bottleneck compute or memory?
③ Profiling	Observe the same workload with ixsmi / ixsys / ixkn	Utilization, trace timeline, kernel-level evidence	Where exactly is it stalling? Does it agree with the Roofline diagnosis?

African Scientific Research and Innovation Council

BAAI 智源研究院

北京大学 PEKING UNIVERSITY

中国人民大学 RENMIN UNIVERSITY OF CHINA

Thanks

Questions & further discussion welcome

Guojie Luo
罗国杰
gluo@pku.edu.cn

Jing Wang
王晶
jwang@ruc.edu.cn

BAAI Team
智源团队
github.com/flagos-ai



Supported by UNDP.